PROYECTO FINAL APLICACIÓN “EGO”

INTEGRANTES: ARAGON VARGAS IRVING JONATHAN

DEL ROSARIO CASTILLO IRVING , JUÁREZ COTERA JOSÉ ARTURO ,

CHAVOLLA VANEGAS JOSÉ FERNANDO , SAAVEDRA ORDOÑEZ JOSÉ ALI

PROFESOR : IVÁN AZAMAR PALMA

MATERIA : PROGRAMACIÓN DE APL PARA DISPOSITIVOS MOVILES

GRUPO : 4951



INDICE

Contenido

INTRODUCCIÓN 3
MARCO TEORICO 5
SQLite 5
SharedPreferences 5
FileStream 5
Menús y Diálogos 5
Mapas 6
GPS 6
Web Services DML 6
PROBLEMÁTICA A RESOLVER 7
OBJETIVO GENERAL 8
OBJETIVOS ESPECIFICOS 8
JUSTIFICACIÓN 9
ALCANCE 10
LIMITACIONES 10
DESARROLLO 11
SQLITE 14
Diálogos y menús 17
SharedPreference 21
FILE STREAM 30
WEB SERVICES DML 31
CONCLUCIÓN 33
BIBLIOGRAFIAS 34

INTRODUCCIÓN

En la actualidad, el uso de aplicaciones móviles orientadas al bienestar físico ha cobrado gran relevancia, debido al creciente interés de las personas por mantener un estilo de vida saludable. La tecnología se ha convertido en una herramienta fundamental para apoyar la actividad física, la organización de rutinas y el seguimiento del progreso personal. En este contexto, el desarrollo de aplicaciones móviles enfocadas al ejercicio no solo responde a una necesidad social, sino que también representa una oportunidad para integrar conocimientos tecnológicos con soluciones prácticas que impacten de manera positiva en la vida cotidiana de los usuarios.

El presente proyecto consiste en el desarrollo de una aplicación móvil dirigida a personas que realizan actividad física, cuyo objetivo principal es facilitar la planificación, el control y el seguimiento de hábitos relacionados con el ejercicio y la alimentación. La aplicación permitirá a los usuarios crear sus propias rutinas de entrenamiento y organizarlas de acuerdo con los días de la semana en que desean realizarlas, promoviendo así una mayor constancia y disciplina en sus actividades. Asimismo, contará con un apartado dedicado a la gestión de dietas, en el cual el usuario podrá tanto registrar planes alimenticios personalizados como consultar dietas previamente establecidas dentro de la aplicación.

Otro de los aspectos fundamentales de la aplicación es el seguimiento del progreso físico. Para ello, se integrará un módulo que mostrará gráficas visuales sobre los avances y mejoras del usuario a lo largo del tiempo, permitiendo una interpretación clara y motivadora de sus resultados. De igual manera, se incorporará una sección con mapas interactivos que facilitará la creación de rutas para correr, mostrando la distancia recorrida en kilómetros y un contador de tiempo que registre la duración de la actividad, brindando así una experiencia más completa y funcional.

Desde el punto de vista técnico, la aplicación será desarrollada en Android Studio utilizando el lenguaje de programación Kotlin, lo que permitirá aprovechar las ventajas de un entorno moderno, seguro y eficiente para el desarrollo móvil. Además, se implementarán diversas tecnologías y herramientas fundamentales, tales como SQLite para el almacenamiento local de datos, SharedPreferences para la gestión de configuraciones del usuario, FileStream para el manejo de archivos, menús y diálogos para una mejor interacción con la interfaz, así como la integración de mapas y GPS para la localización y trazado de rutas. Finalmente, el uso de Web Services DML permitirá la comunicación con servicios externos, fortaleciendo la funcionalidad y escalabilidad del sistema.

En conjunto, este proyecto busca no solo ofrecer una aplicación funcional e intuitiva, sino también demostrar la aplicación práctica de distintos conceptos de desarrollo

de software, contribuyendo al aprendizaje integral y a la creación de una herramienta tecnológica con impacto real en el bienestar de las personas.

MARCO TEORICO

SQLite

SQLite es un sistema de gestión de bases de datos relacional ligero que se integra directamente dentro de las aplicaciones móviles, sin necesidad de un servidor externo. En el contexto del desarrollo de aplicaciones Android, SQLite permite almacenar información estructurada de forma local en el dispositivo del usuario, como rutinas de ejercicio, dietas personalizadas, registros de progreso y datos históricos de actividad. Su uso favorece la persistencia de la información aun cuando la aplicación no cuenta con conexión a internet, lo cual mejora la experiencia del usuario y garantiza la disponibilidad de los datos en todo momento. Además, al tratarse de una base de datos eficiente y segura, contribuye a un manejo adecuado de la información dentro de la aplicación.

SharedPreferences

SharedPreferences es un mecanismo de almacenamiento que se utiliza para guardar datos simples en formato clave-valor. Generalmente se emplea para conservar configuraciones del usuario, preferencias de uso o información básica, como el nombre del usuario, el tema de la aplicación o parámetros de personalización. En este proyecto, su implementación resulta fundamental para mejorar la experiencia del usuario, ya que permite que la aplicación recuerde ciertos ajustes incluso después de cerrarse, brindando así una interacción más personalizada y continua.

FileStream

El uso de FileStream en Android permite la creación, lectura y escritura de archivos dentro del almacenamiento del dispositivo. Esta funcionalidad resulta útil cuando se requiere guardar información más extensa o generar respaldos de datos, como reportes de progreso, exportación de rutinas o almacenamiento de archivos generados por el usuario. La implementación de FileStream fortalece la gestión de la información, proporcionando mayor flexibilidad en el manejo de datos y ampliando las capacidades funcionales de la aplicación.

Menús y Diálogos

Los menús y diálogos son elementos esenciales dentro del diseño de interfaces gráficas en aplicaciones móviles. Los menús permiten organizar las funcionalidades de forma clara y accesible, facilitando la navegación del usuario entre las distintas secciones de la aplicación, como rutinas, dietas, progreso y mapas. Por su parte, los diálogos sirven para mostrar mensajes importantes, solicitar confirmaciones o

capturar información relevante. Su adecuada implementación contribuye a una experiencia de usuario más intuitiva, clara y amigable.

Mapas

La integración de mapas digitales dentro de aplicaciones móviles se ha convertido en una herramienta clave para ofrecer servicios basados en localización. En este proyecto, los mapas permiten al usuario visualizar rutas para correr, planificar recorridos y conocer la distancia aproximada entre puntos. Gracias a esta funcionalidad, la aplicación no solo apoya la organización del ejercicio, sino que también fomenta la exploración de espacios y el uso consciente del entorno físico.

GPS

El Sistema de Posicionamiento Global (GPS) es una tecnología que posibilita la obtención de la ubicación geográfica en tiempo real. En aplicaciones orientadas al ejercicio, el GPS cumple un papel fundamental, ya que permite registrar distancias recorridas, velocidad, tiempo y ubicación durante actividades como correr o caminar. Su implementación aporta precisión al seguimiento del progreso físico y proporciona datos confiables que pueden ser utilizados para generar estadísticas y gráficas de desempeño.

Web Services DML

Los Web Services permiten la comunicación entre la aplicación móvil y sistemas externos a través de internet. Mediante el uso de operaciones DML (Data Manipulation Language), como inserción, actualización, consulta y eliminación de datos, es posible gestionar información almacenada en servidores remotos. Esto favorece la escalabilidad de la aplicación, ya que permite sincronizar datos, realizar respaldos en la nube y ofrecer servicios más robustos. Además, contribuye a que la aplicación pueda evolucionar hacia un sistema más completo y conectado.

En conjunto, estos componentes tecnológicos constituyen la base teórica y práctica sobre la cual se construye la aplicación, garantizando su funcionalidad, eficiencia y utilidad dentro del ámbito del desarrollo móvil orientado al bienestar físico.

PROBLEMÁTICA A RESOLVER

La problemática principal radica en la falta de acompañamiento personalizado e inteligente en muchas aplicaciones de ejercicio, lo que provoca que los usuarios realicen movimientos incorrectos, se desmotiven y no logren dar seguimiento efectivo a su progreso. Además, la escasa integración entre planificación, monitoreo y retroalimentación limita el aprovechamiento de la tecnología como apoyo real para mejorar la condición física y prevenir lesiones.

OBJETIVO GENERAL

Desarrollar una aplicación móvil para Android que apoye a las personas en la planificación, seguimiento y mejora de su actividad física mediante herramientas tecnológicas e inteligencia artificial, ofreciendo una experiencia personalizada, segura y eficiente.

OBJETIVOS ESPECIFICOS

1. Utilizar tecnologías como SQLite, SharedPreferences y FileStream para garantizar el almacenamiento adecuado de la información.
2. Incorporar una inteligencia artificial capaz de apoyar al usuario en la ejecución de ejercicios, detectando errores mediante la cámara y mostrando la forma correcta de realizarlos.
3. Mejorar la experiencia de usuario mediante una interfaz intuitiva basada en menús y diálogos claros.
4. Integrar mapas y GPS para la creación de rutas de ejercicio, medición de distancia y control del tiempo de actividad.

JUSTIFICACIÓN

El desarrollo de esta aplicación se justifica por la creciente necesidad de contar con herramientas tecnológicas que promuevan estilos de vida saludables y apoyen a las personas en el cuidado de su salud física. Actualmente, muchas personas desean realizar ejercicio pero no cuentan con la orientación adecuada, ya sea por falta de tiempo, recursos económicos o acceso a entrenadores profesionales. Ante esta situación, una aplicación móvil con funciones inteligentes representa una alternativa accesible y eficiente.

Desde el punto de vista social, el proyecto contribuye a fomentar hábitos saludables, prevenir problemas de salud asociados al sedentarismo y mejorar la calidad de vida de los usuarios. La incorporación de inteligencia artificial para la corrección de ejercicios agrega un valor significativo, ya que permite reducir el riesgo de lesiones y aumentar la confianza del usuario al entrenar.

Desde el ámbito académico y tecnológico, el proyecto permite aplicar conocimientos teóricos y prácticos relacionados con el desarrollo de aplicaciones móviles, bases de datos, almacenamiento, servicios web y tecnologías emergentes como la inteligencia artificial. Esto fortalece las competencias profesionales y demuestra la capacidad de crear soluciones innovadoras con impacto real.

ALCANCE

La aplicación permitirá a los usuarios gestionar rutinas de ejercicio, registrar dietas, visualizar su progreso mediante gráficas y utilizar mapas para crear rutas de entrenamiento con apoyo de GPS. También incluirá un módulo de inteligencia artificial capaz de brindar orientación básica sobre la ejecución correcta de ciertos ejercicios mediante el uso de la cámara del dispositivo. Asimismo, contará con almacenamiento local y posibilidad de interacción con servicios web para el manejo de información el sistema estará dirigido principalmente a usuarios con dispositivos Android y conocimientos básicos en el uso de aplicaciones móviles, enfocándose en actividades físicas comunes como caminar, correr y ejercicios funcionales.

LIMITACIONES

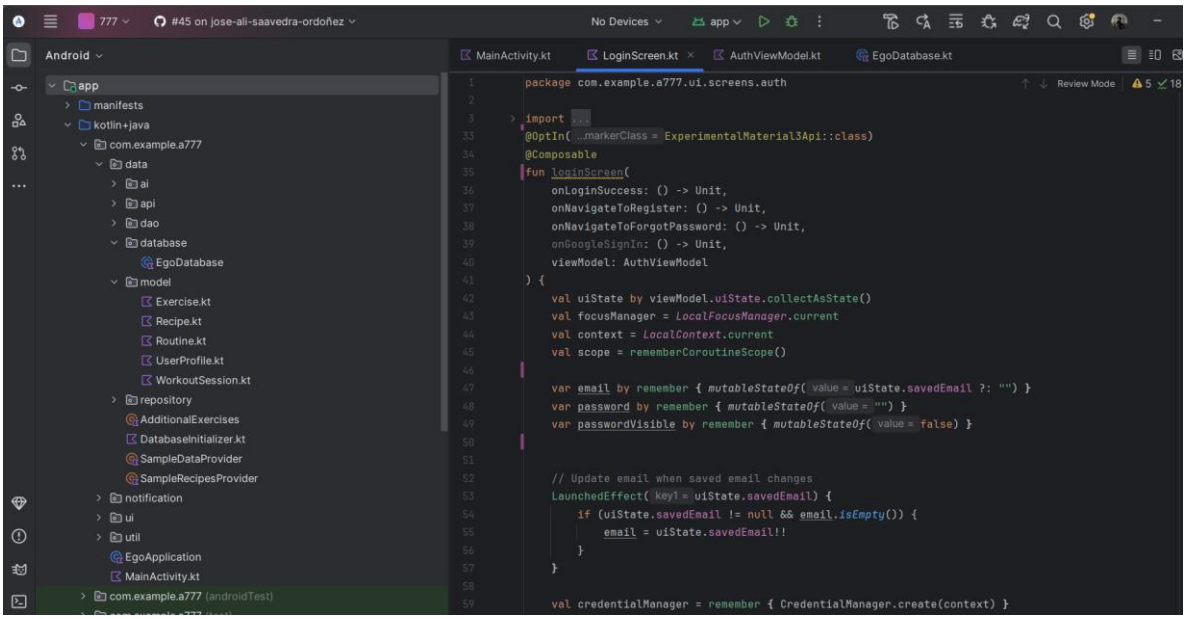
Entre las principales limitaciones se encuentra la dependencia del hardware del dispositivo, ya que el correcto funcionamiento del GPS, la cámara y el procesamiento de la inteligencia artificial puede variar según la calidad del equipo. Además, la retroalimentación proporcionada por la inteligencia artificial no sustituye la asesoría profesional de un entrenador certificado o un especialista en salud , Otra limitación importante es que la aplicación requiere, en algunos casos, conexión a internet para el uso de servicios web y ciertas funciones avanzadas. Asimismo, el reconocimiento y corrección de ejercicios estará limitado a un conjunto específico de movimientos previamente definidos en el sistema, por lo que no abarcará todos los tipos de entrenamiento existentes.

DESARROLLO

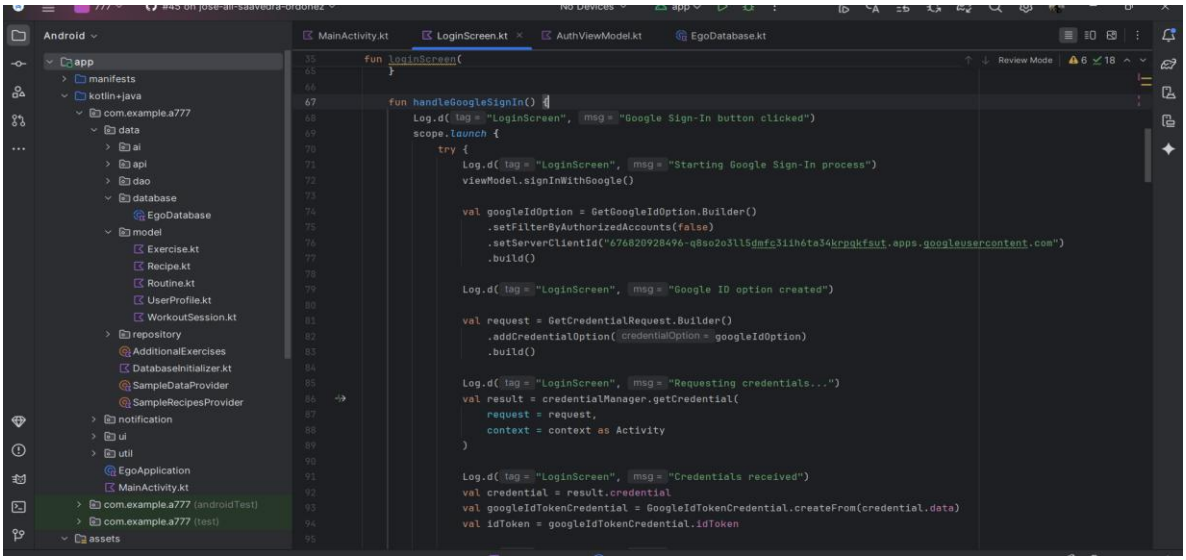
La aplicación de basa en varios puntos que son requeridos por el profesor los cuales se desglosaran en este aparatado.

Antes de empezar con cada uno de los apartados nuestra aplicación empieza por un inicio de sesión.

Se presenta una pantalla de inicio de sesión desarrollada con Jetpack Compose que permite al usuario autenticarse mediante correo y contraseña o con Google, gestionando el estado de la interfaz y la navegación a registro y recuperación de contraseña.



Aquí se muestra que controla el proceso de inicio de sesión con Google dentro de la pantalla de autenticación, utilizando Credential Manager para solicitar las credenciales del usuario, obtener el token de Google de forma segura y delegar al ViewModel la validación del acceso y la continuación de la navegación en la aplicación.



```
fun loginScreen(
    )

fun handleGoogleSignIn() {
    Log.d( tag = "LoginScreen", msg = "Google Sign-In button clicked")
    scope.launch {
        try {
            Log.d( tag = "LoginScreen", msg = "Starting Google Sign-In process")
            viewModel.signInWithGoogle()

            val googleIdOption = GetGoogleIdOption.Builder()
                .setFilterByAuthorizedAccounts(false)
                .setServerClientId("676820928496-q8so2o3ll5dmfc3iih6ta34krpgkfsut.apps.googleusercontent.com")
                .build()

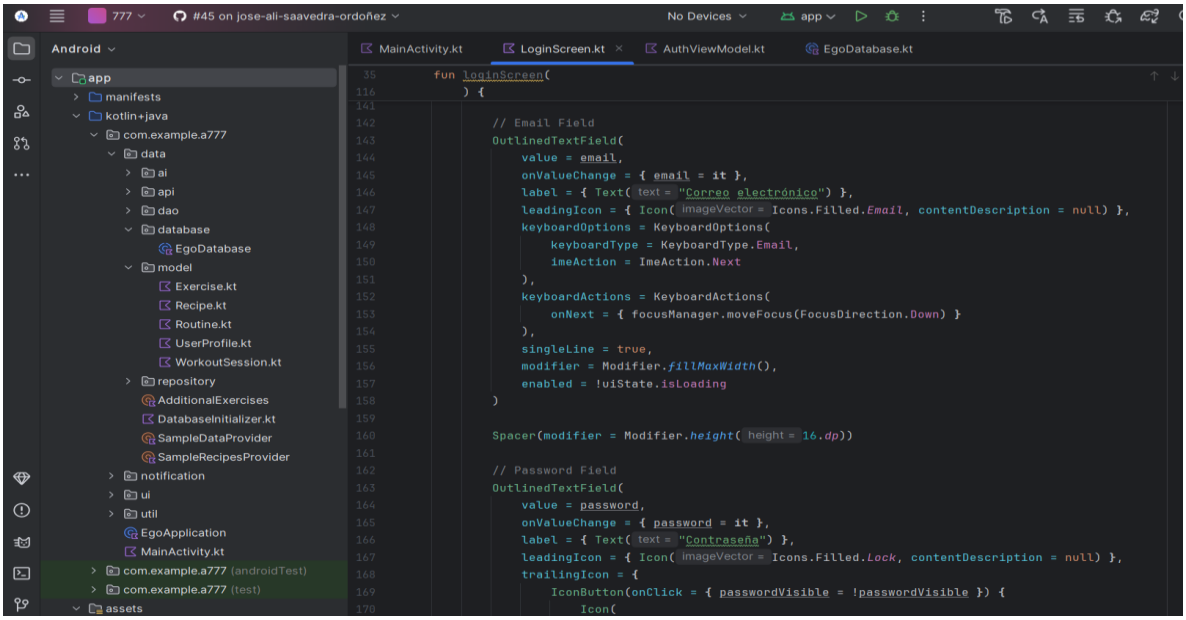
            Log.d( tag = "LoginScreen", msg = "Google ID option created")

            val request = GetCredentialRequest.Builder()
                .addCredentialOption( credentialOption = googleIdOption)
                .build()

            Log.d( tag = "LoginScreen", msg = "Requesting credentials...")
            val result = credentialManager.getCredential(
                request = request,
                context = context as Activity
            )

            Log.d( tag = "LoginScreen", msg = "Credentials received")
            val credential = result.credential
            val googleIdTokenCredential = GoogleIdTokenCredential.createFrom(credential.data)
            val idToken = googleIdTokenCredential.idToken
```

Campos de texto para correo electrónico y contraseña desarrollados con OutlinedTextField, que permiten al usuario ingresar sus credenciales de forma clara y segura. El campo de correo valida el tipo de teclado y gestiona el cambio de enfoque entre campos, mientras que el campo de contraseña incorpora iconos, control de visibilidad y deshabilita la interacción durante el proceso de autenticación, mejorando la usabilidad y evitando errores mientras se carga la información.



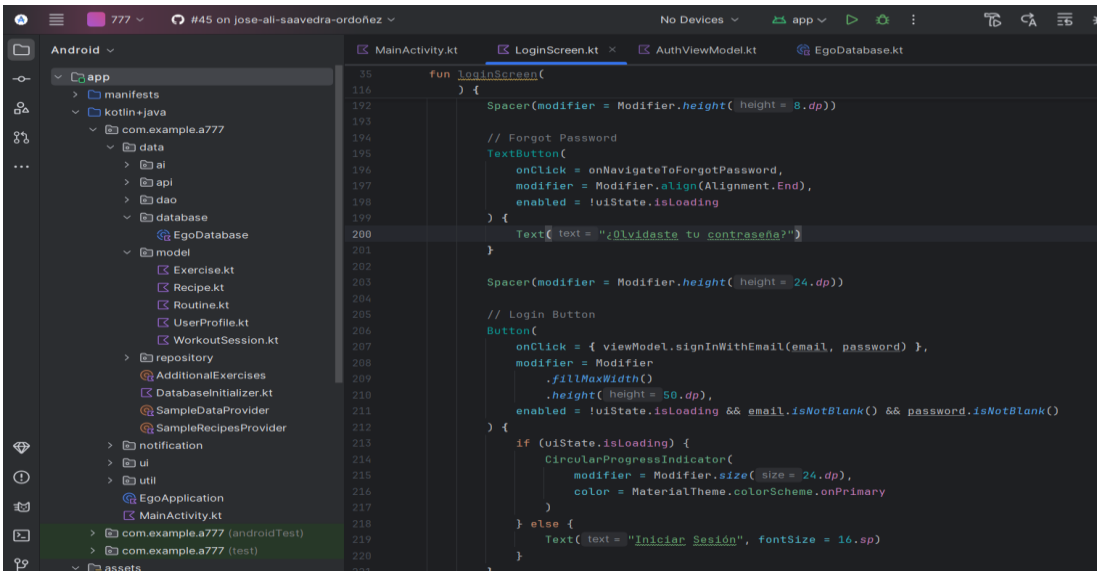
```
fun loginScreen(
    ) {

        // Email Field
        OutlinedTextField(
            value = email,
            onValueChange = { email = it },
            label = { Text( text = "Correo electrónico") },
            leadingIcon = { Icon( imageVector = Icons.Filled.Email, contentDescription = null) },
            keyboardOptions = KeyboardOptions(
                keyboardType = KeyboardType.Email,
                imeAction = ImeAction.Next
            ),
            keyboardActions = KeyboardActions(
                onNext = { focusManager.moveFocus(FocusDirection.Down) }
            ),
            singleLine = true,
            modifier = Modifier.fillMaxWidth(),
            enabled = !uiState.isLoading
        )

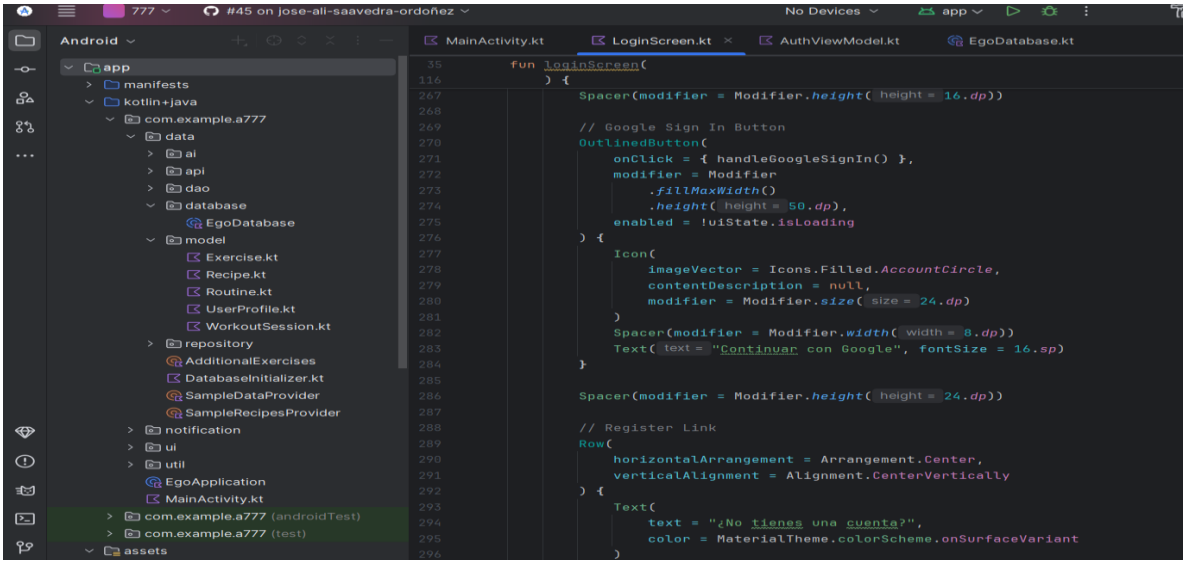
        Spacer(modifier = Modifier.height( height = 16.dp))

        // Password Field
        OutlinedTextField(
            value = password,
            onValueChange = { password = it },
            label = { Text( text = "Contraseña") },
            leadingIcon = { Icon( imageVector = Icons.Filled.Lock, contentDescription = null) },
            trailingIcon = {
                IconButton(onClick = { passwordVisible = !passwordVisible }) {
                    Icon(
```

Sección de acciones de autenticación que incluye un botón para recuperación de contraseña y el botón principal de inicio de sesión. El enlace de “¿Olvidaste tu contraseña?” permite al usuario navegar al flujo de restablecimiento, mientras que el botón de inicio de sesión valida que los campos estén completos, gestiona el estado de carga y muestra un indicador de progreso para evitar múltiples intentos mientras se procesa la autenticación.

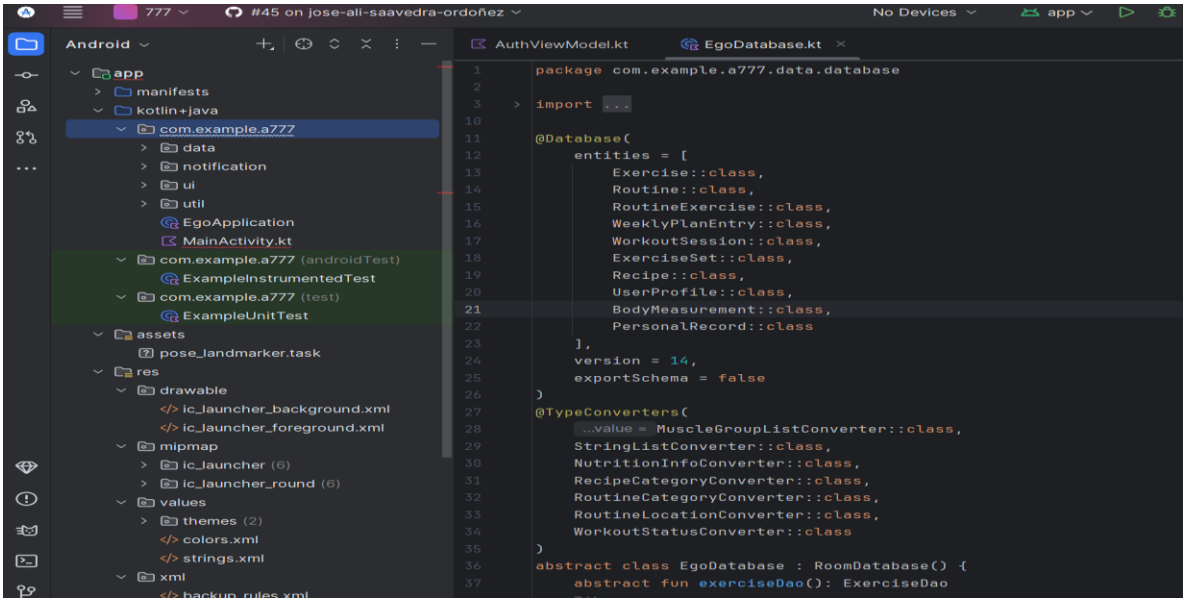


Botón que permite iniciar sesión rápidamente con una cuenta de Google, simplificando el acceso a la aplicación sin necesidad de crear una contraseña. Debajo, se muestra un enlace que dirige al usuario al registro para crear una nueva cuenta en caso de no contar con una.

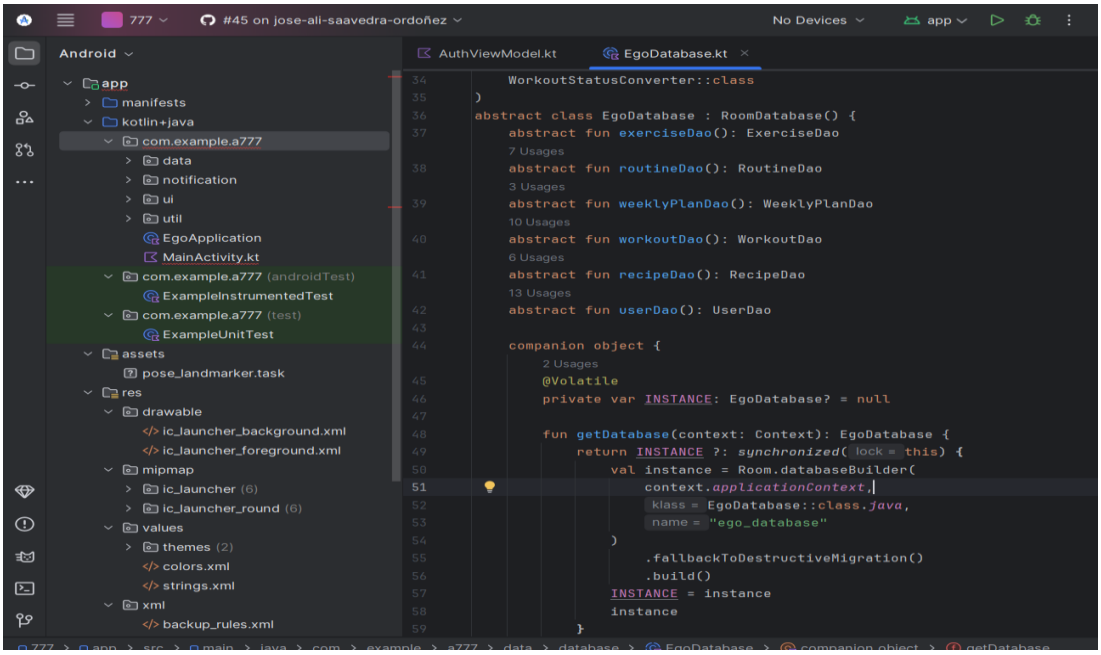


SQLITE

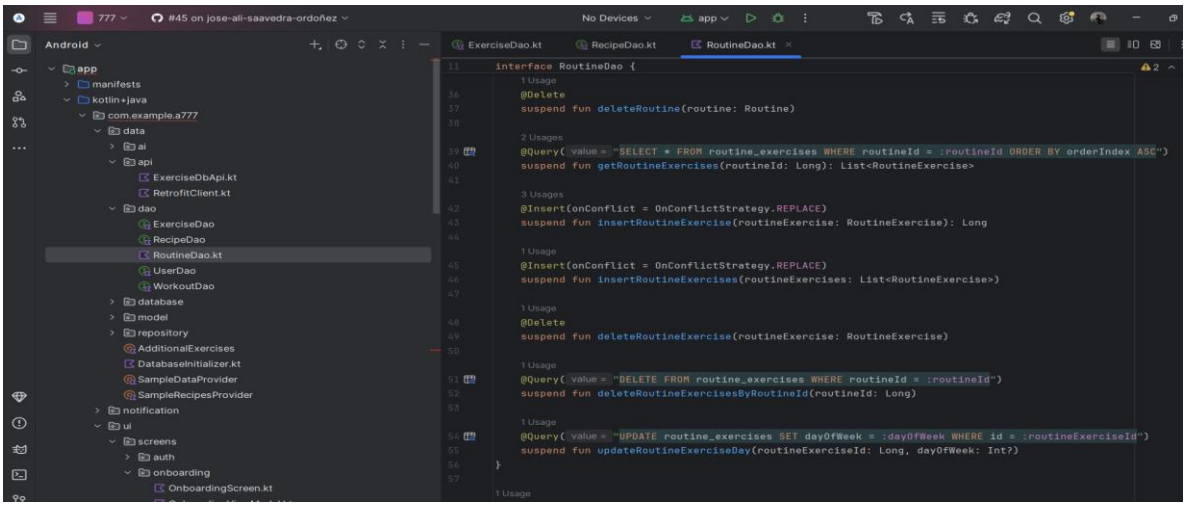
Archivo que define la base de datos local de la aplicación, donde se guardan de forma segura los ejercicios, rutinas, recetas y datos del usuario. Utiliza Room para organizar la información en tablas y permitir que la app acceda a los datos incluso sin conexión a internet.



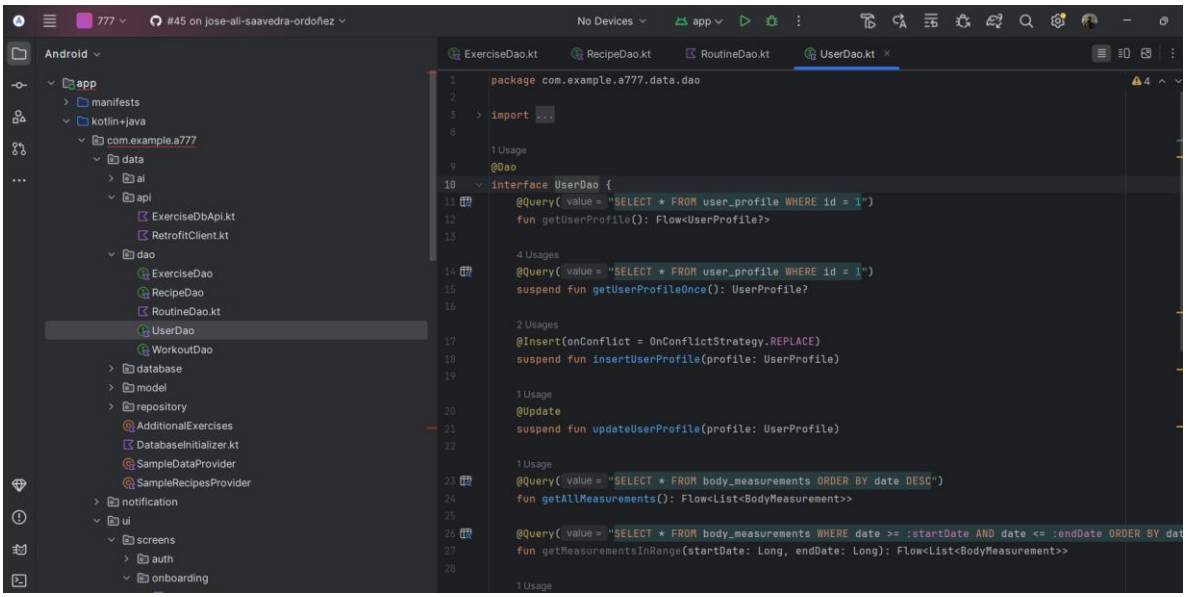
Esta sección define cómo la aplicación accede a la base de datos mediante distintos DAOs para manejar ejercicios, rutinas, entrenamientos, recetas y datos del usuario. Además, garantiza que la base de datos se cree una sola vez y se comparta en toda la app, asegurando un uso eficiente de la memoria y un acceso seguro a la información almacenada en el dispositivo.



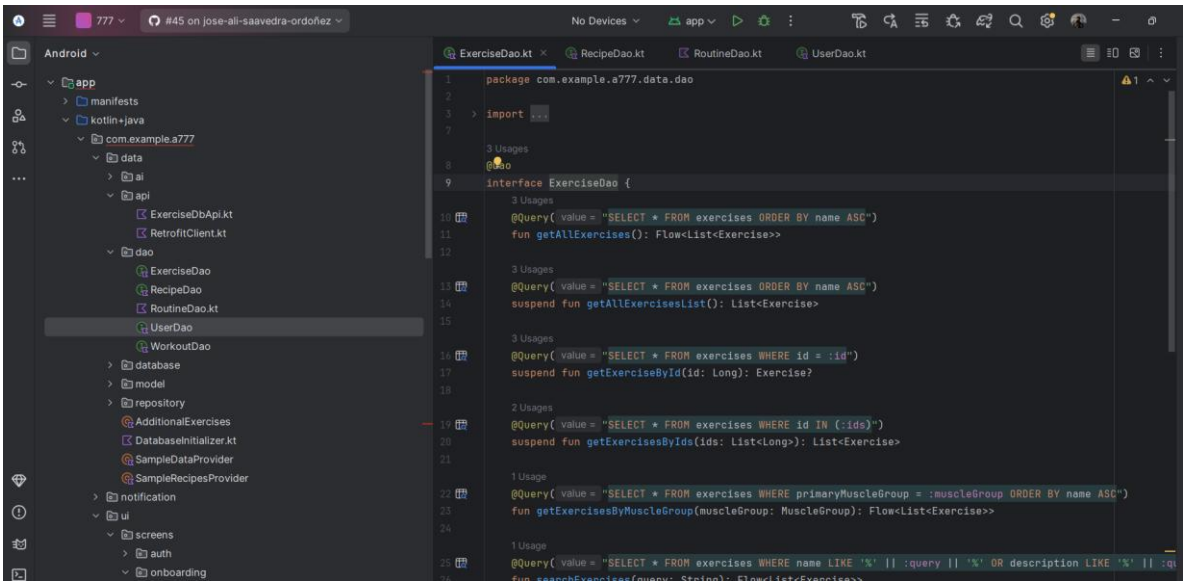
Conjunto de funciones del DAO encargado de gestionar las rutinas y sus ejercicios dentro de la base de datos. Permite eliminar rutinas, consultar los ejercicios asociados a cada rutina, insertar o actualizar ejercicios, eliminarlos individualmente o por rutina, y modificar el día de la semana en que se realizan, asegurando un control ordenado y persistente de la planificación de entrenamientos del usuario.



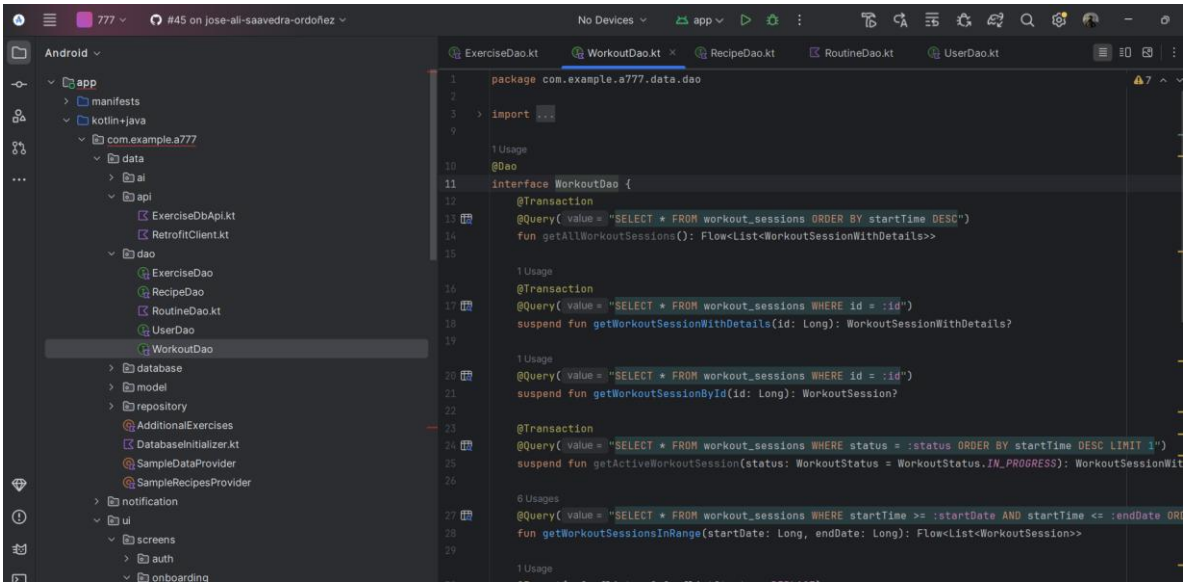
DAO encargado de gestionar la información del perfil del usuario y sus registros físicos dentro de la base de datos. Permite consultar y actualizar los datos personales, así como registrar y obtener mediciones corporales a lo largo del tiempo, facilitando el seguimiento del progreso del usuario de manera ordenada y persistente.



DAO responsable de gestionar los ejercicios almacenados en la aplicación. Permite obtener la lista completa de ejercicios, consultar ejercicios específicos, filtrar por grupo muscular y realizar búsquedas por nombre o descripción, facilitando al usuario encontrar y organizar sus rutinas de entrenamiento de forma eficiente.

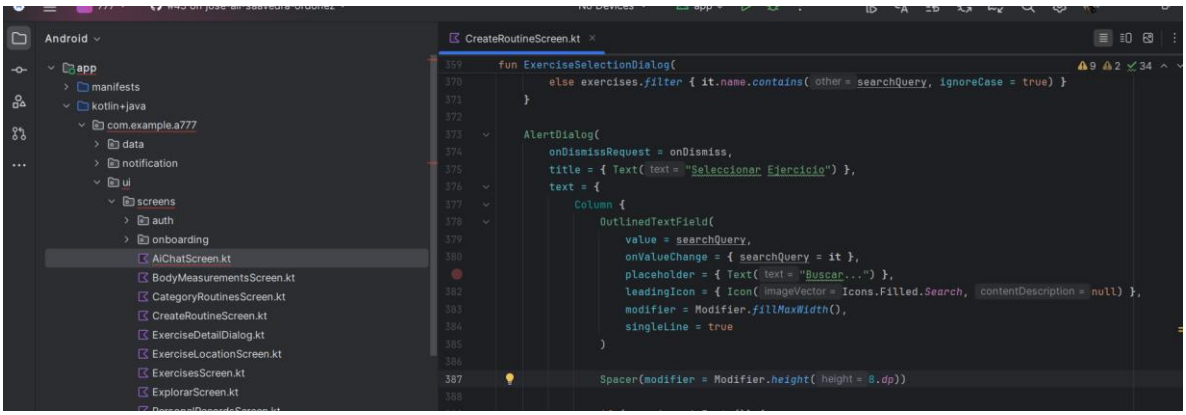


DAO encargado de gestionar las sesiones de entrenamiento del usuario. Permite registrar nuevas sesiones, consultar el historial de entrenamientos, obtener el entrenamiento activo y acceder a los detalles completos de cada sesión, facilitando el seguimiento del progreso y la organización de la actividad física dentro de la aplicación.

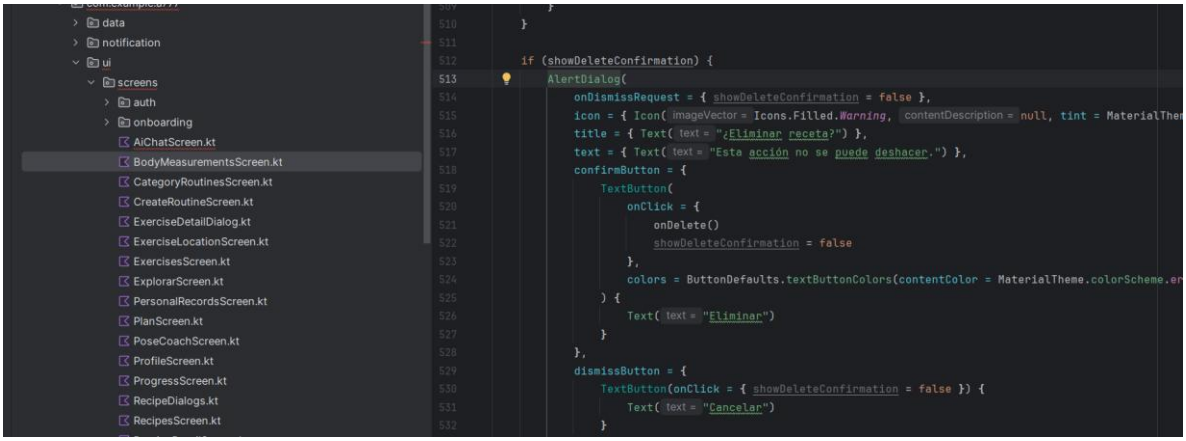


Diálogos y menús

Este diálogo emergente permite al usuario buscar y seleccionar un ejercicio de forma sencilla. Incluye un campo de búsqueda que filtra los ejercicios conforme el usuario escribe, mejorando la rapidez para encontrar el ejercicio deseado. Mientras la lista de ejercicios se carga o no hay resultados disponibles.

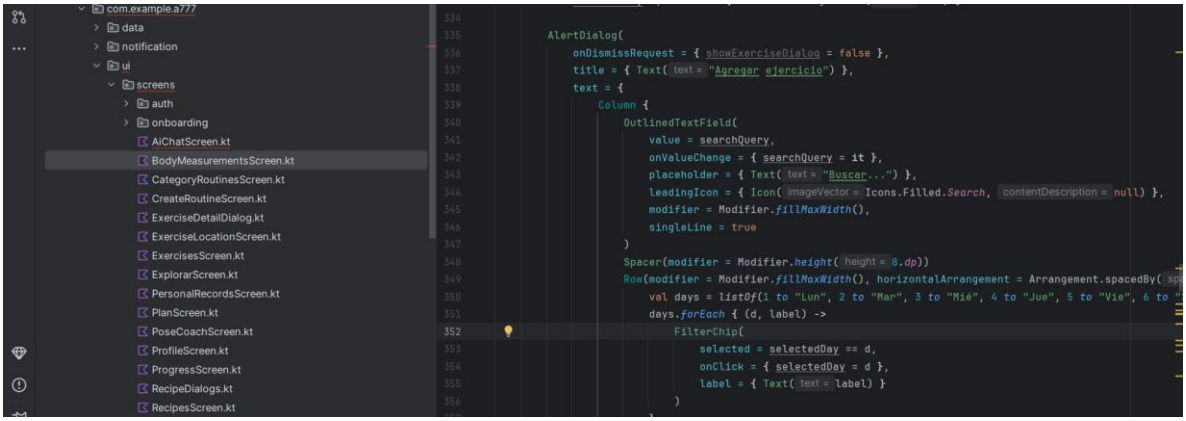


Este diálogo de confirmación se muestra cuando el usuario intenta eliminar una receta. Presenta un mensaje de advertencia claro indicando que la acción es irreversible, ayudando a evitar eliminaciones accidentales. Incluye dos opciones: Eliminar, que confirma la acción y ejecuta el borrado del contenido, y Cancelar, que cierra el diálogo sin realizar cambios.

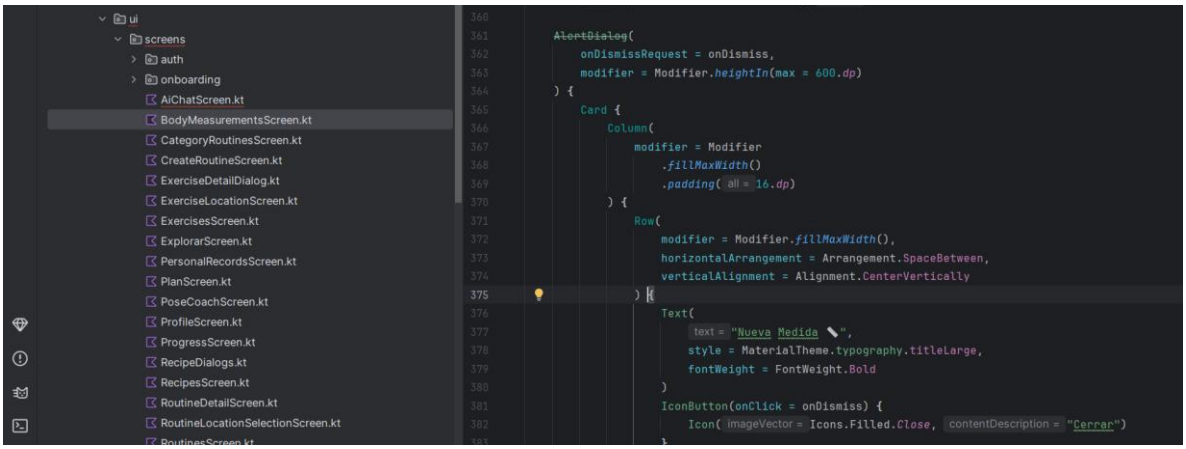


Este diálogo permite al usuario agregar un ejercicio a su rutina de forma guiada. Incluye un campo de búsqueda para filtrar ejercicios rápidamente y facilitar su

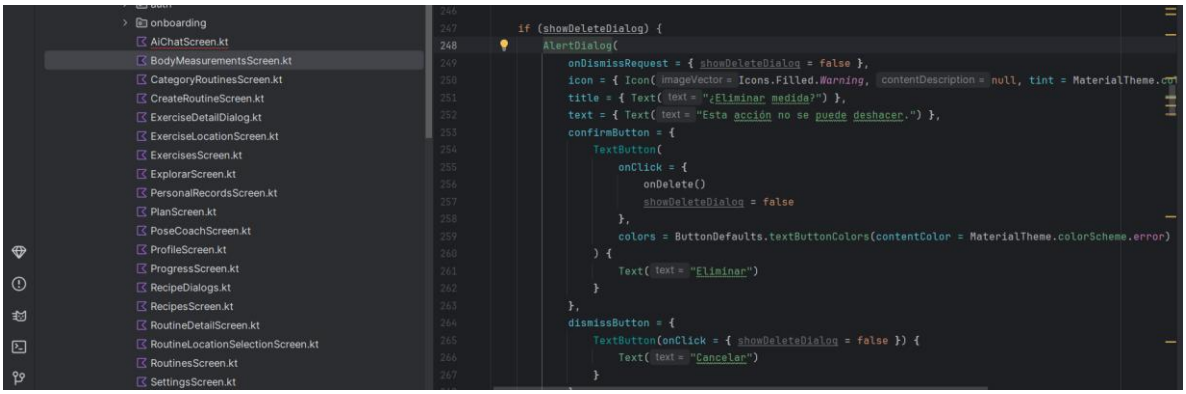
localización. Además, muestra un conjunto de opciones por día de la semana, permitiendo seleccionar cuándo se realizará el ejercicio.



Este diálogo se utiliza para registrar una nueva medida corporal. Se presenta como una ventana emergente con un tamaño máximo definido para mantener una buena visualización en pantalla. Incluye un título claro y un botón de cierre, permitiendo al usuario cancelar la acción en cualquier momento.

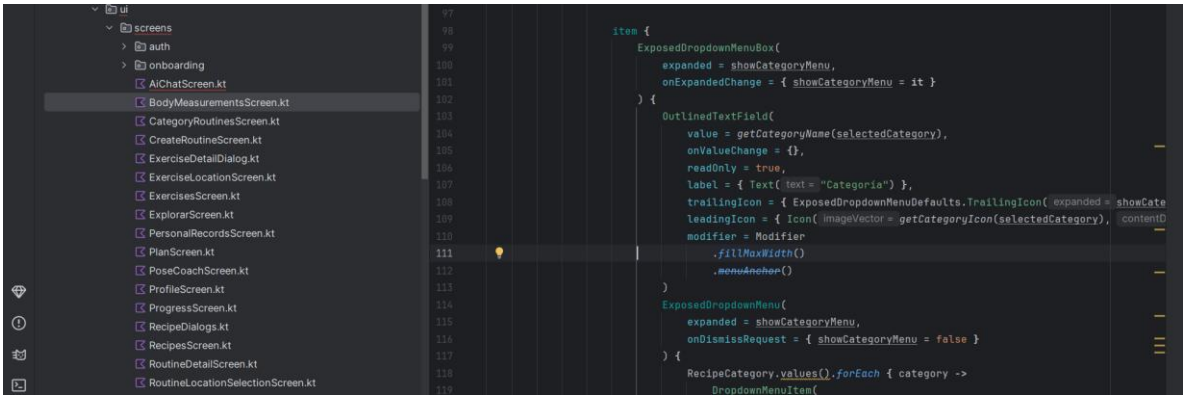


Este diálogo de confirmación se muestra cuando el usuario intenta eliminar una medida registrada. Presenta una advertencia visual mediante un ícono de alerta y un mensaje claro que informa que la acción es irreversible. Ofrece dos opciones: confirmar la eliminación o cancelar la operación, evitando borrados accidentales y garantizando una interacción segura y consciente por parte del usuario.



Menús

Este componente corresponde a un menú desplegable de selección que permite al usuario elegir una categoría de una lista de opciones. El campo de texto es de solo lectura y funciona como disparador del menú al tocarlo, se despliega una lista con las categorías disponibles.



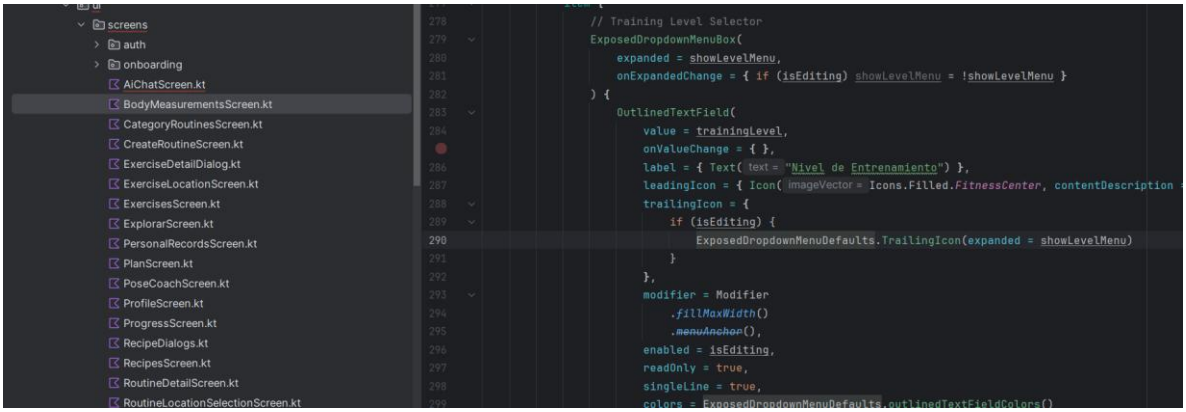
Este menú desplegable muestra una lista de categorías disponibles para que el usuario seleccione una opción. Cada elemento del menú incluye un ícono y un texto descriptivo, lo que facilita su identificación visual.



Este componente muestra un campo de selección con menú desplegable para elegir una categoría. El campo es de solo lectura y actúa como un botón que, al tocarse, despliega un menú con las opciones disponibles. Incluye un ícono representativo de la categoría seleccionada y un indicador visual que muestra si el menú está abierto o cerrado.



Este elemento presenta un menú desplegable para seleccionar el nivel de entrenamiento del usuario. El campo se muestra como un texto de solo lectura y solo es interactivo cuando el modo de edición está habilitado. Incluye un ícono representativo y un indicador visual que señala la posibilidad de desplegar el menú



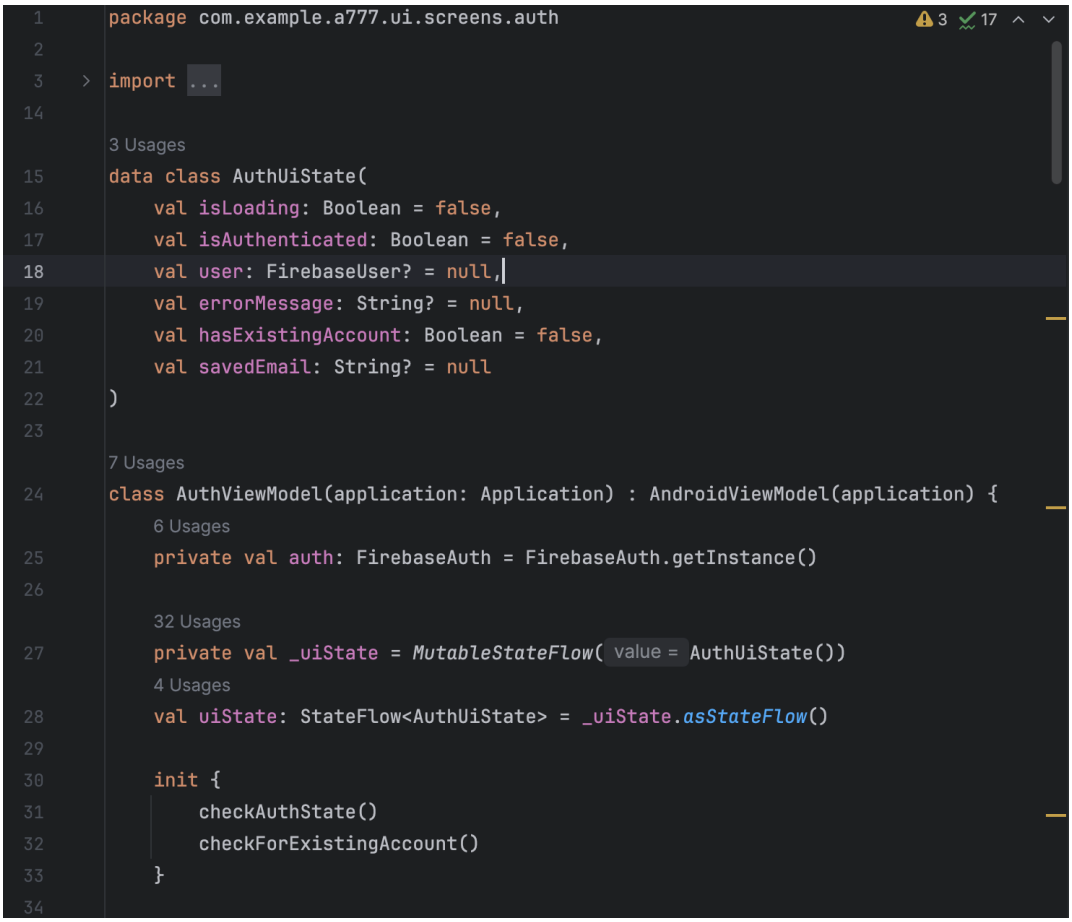
Este componente muestra un campo con menú desplegable para seleccionar el nivel de entrenamiento. El usuario puede interactuar con él únicamente cuando la

edición está habilitada, lo que evita cambios accidentales. El campo presenta un ícono representativo y un indicador visual que señala si el menú está desplegado



SharedPreference

Lo siguiente que vamos a explicar va ser nuestro sahared preference de la aplicación el cual nos ayuda lo cual usamos varias librerias la principal viewmodel En este ViewModel se usa SharedPreferences para guardar y recuperar el último correo electrónico utilizado



```
package com.example.a777.ui.screens.auth                    ⚠ 3  ✓ 17

import ...

3 Usages
data class AuthUiState(
    val isLoading: Boolean = false,
    val isAuthenticated: Boolean = false,
    val user: FirebaseUser? = null,
    val errorMessage: String? = null,
    val hasExistingAccount: Boolean = false,
    val savedEmail: String? = null
)

7 Usages
class AuthViewModel(application: Application) : AndroidViewModel(application) {
    6 Usages
    private val auth: FirebaseAuth = FirebaseAuth.getInstance()

    32 Usages
    private val _uiState = MutableStateFlow( value = AuthUiState())
    4 Usages
    val uiState: StateFlow<AuthUiState> = _uiState.asStateFlow()

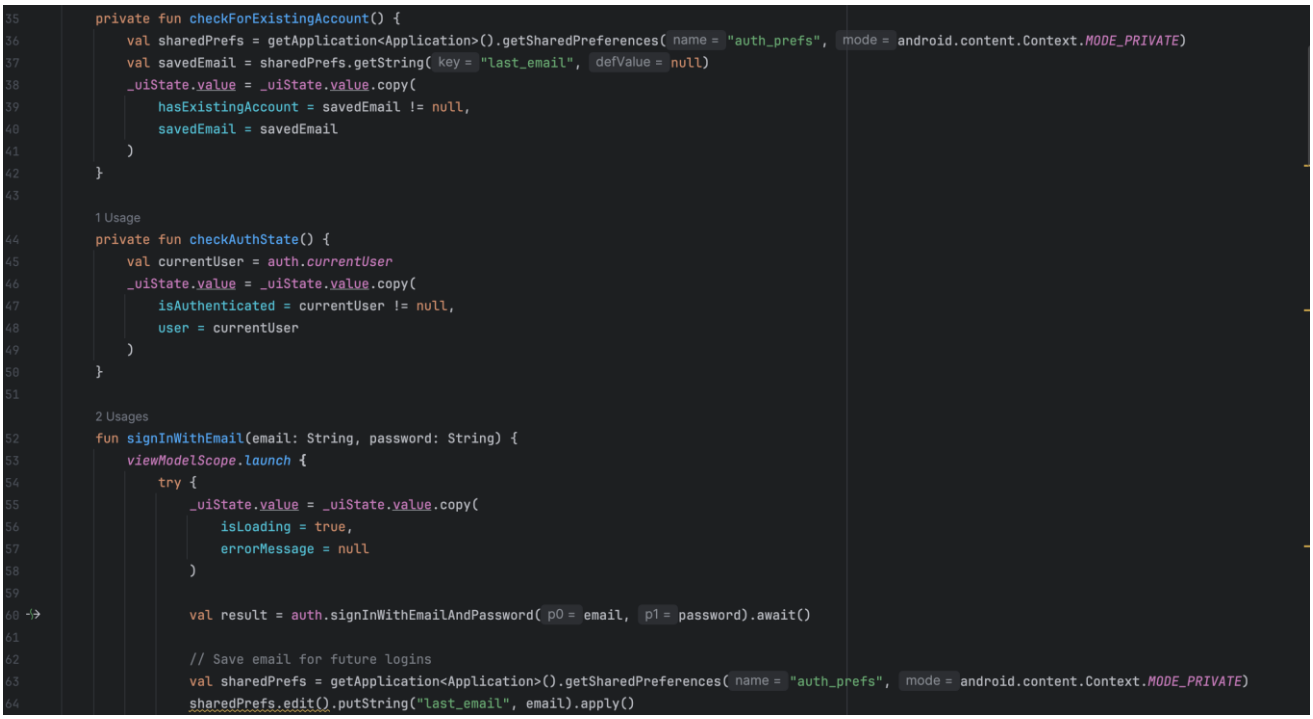
    init {
        checkAuthState()
        checkForExistingAccount()
    }
```



En esta imagen se puede ver como nos aparece el login para poner nuestro correo electronico y contraseña que no pueden repetirse por ningun otro usuario

Despues ya vamos con la creaccion de nuestro shared preference para las cuentas de cada uno de los usuarios con el cual el objetivo principales es

Detectar si ya hubo una cuenta usada antes en el dispositivo , Facilitar el inicio de sesión al usuario y Recordar información básica entre ejecuciones de la app



```
    private fun checkForExistingAccount() {
        val sharedPrefs = getApplication<Application>().getSharedPreferences( name = "auth_prefs", mode = android.content.Context.MODE_PRIVATE)
        val savedEmail = sharedPrefs.getString( key = "last_email", defValue = null)
        _uiState.value = _uiState.value.copy(
            hasExistingAccount = savedEmail != null,
            savedEmail = savedEmail
        )
    }

    1 Usage
    private fun checkAuthState() {
        val currentUser = auth.currentUser
        _uiState.value = _uiState.value.copy(
            isAuthenticated = currentUser != null,
            user = currentUser
        )
    }

    2 Usages
    fun signInWithEmail(email: String, password: String) {
        viewModelScope.launch {
            try {
                _uiState.value = _uiState.value.copy(
                    isLoading = true,
                    errorMessage = null
                )

                val result = auth.signInWithEmailAndPassword( p0 = email, p1 = password).await()

                // Save email for future logins
                val sharedPrefs = getApplication<Application>().getSharedPreferences( name = "auth_prefs", mode = android.content.Context.MODE_PRIVATE)
                sharedPrefs.edit().putString("last_email", email).apply()
```

Lo cual esto hace que leea los datos guardados de la cuenta del usuario al iniciar el viewmodel el cuel en este caso seria el correo electronico del usuario para que ya no se pueda repetir y crear otra cuenta

```
                _uiState.value = _uiState.value.copy(
                    isLoading = false,
                    isAuthenticated = true,
                    user = result.user,
                    errorMessage = null,
                    hasExistingAccount = true,
                    savedEmail = email
                )
            } catch (e: Exception) {
                _uiState.value = _uiState.value.copy(
                    isLoading = false,
                    isAuthenticated = false,
                    errorMessage = getErrorMessage( exception = e)
                )
            }
        }
    }

    2 Usages
    fun signUpWithEmail(email: String, password: String, name: String) {
        viewModelScope.launch {
            try {
                _uiState.value = _uiState.value.copy(
                    isLoading = true,
                    errorMessage = null
                )
```



Como se muestra nuestro error al no ser correcto alguno de los datos ya dados por el usuario el cual si esta mal arojara uno de esos mensajes de error

Esta parte lo que hacer es si las cosas salieron bien a la de guardar cada uno de los datos del usuario el primero de que todo salio bien con mensaje ty el segundo bloque para si saliera algun error en los datos y mandaria un mensaje de error

```
    // Save email for future logins
    val sharedPrefs = getApplication<Application>().getSharedPreferences( name = "auth_prefs",  mode = android.content.Context.MODE_PRIVATE
    sharedPrefs.edit().putString("last_email", email).apply()

    _uiState.value = _uiState.value.copy(
        isLoading = false,
        isAuthenticated = true,
        user = result.user,
        errorMessage = null,
        hasExistingAccount = true,
        savedEmail = email
    )
} catch (e: Exception) {
    _uiState.value = _uiState.value.copy(
        isLoading = false,
        isAuthenticated = false,
        errorMessage = getErrorMessage( exception = e)
    )
```



Asi es nos mostraria alguno de los errores que tenemos y los mensajes que mostraria al haber lago mal como que este en uso o sea invalido

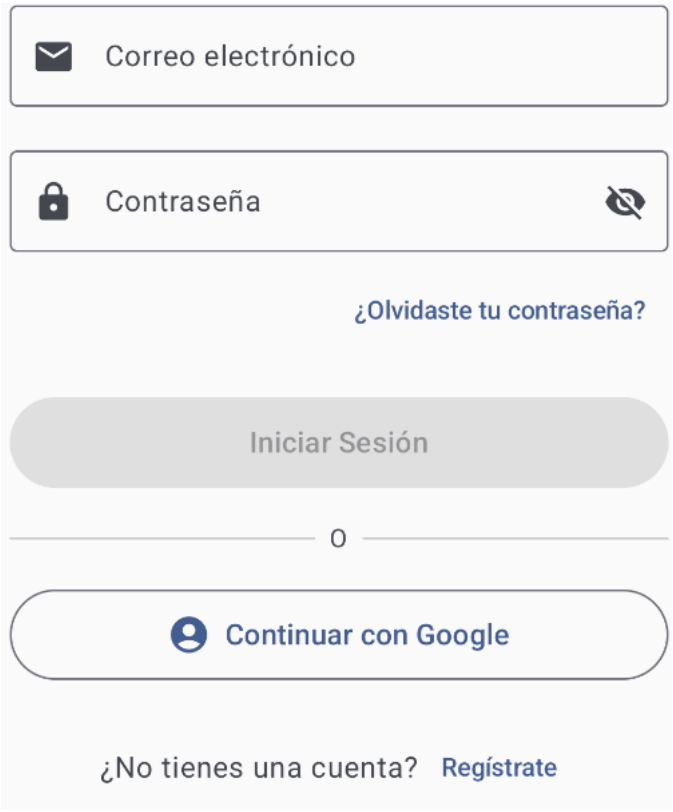
Este apartado es para cuando en algún futuro volvamos a iniciar sección en la aplicación nuevamente y esta ya tenga nuestra cuenta antes echa y también para poder iniciar sesión sin ser nuevo usuario

```
val result = auth.signInWithCredential( p0 = credential).await()

// Save email for future logins (Google account)
val sharedPrefs = getApplication<Application>().getSharedPreferences( name = "auth_prefs",  mode = android.content.Context.MODE_PRIVATE)
sharedPrefs.edit().putString("last_email", result.user?.email).apply()

_uiState.value = _uiState.value.copy(
    isLoading = false,
    isAuthenticated = true,
    user = result.user,
    errorMessage = null,
    hasExistingAccount = true,
    savedEmail = result.user?.email
)
catch (e: Exception) {
_uiState.value = _uiState.value.copy(
    isLoading = false,
    isAuthenticated = false,
    errorMessage = getErrorMessage( exception = e)
)
```

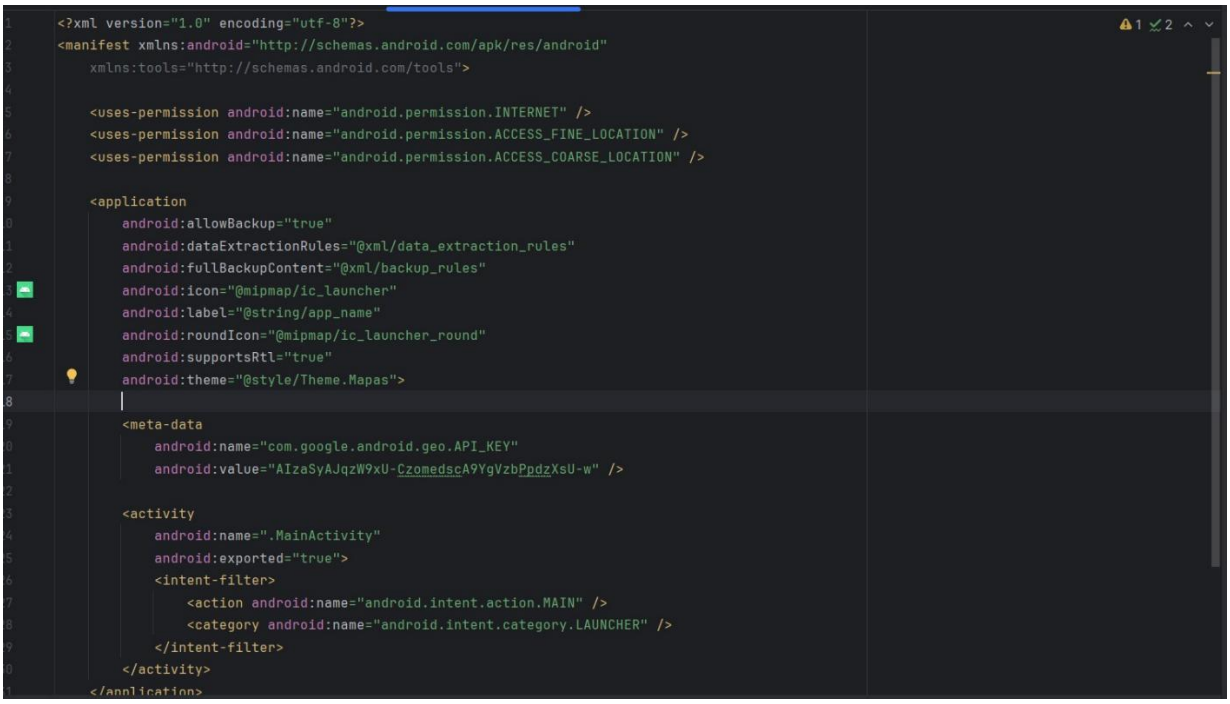
En esta parte lo que podemos ver lo que hace es la autenticacion de google para inicio de secion del usuario el cual igual nos dira si el usuario con unn token que da eso dira si inicio con google o no , Esto guarda el correo del usuario de Google en el almacenamiento local del teléfono. Sirve para:Recordar qué usuario usó la app y Mostrar su correo automáticamente en futuros logins



Asi como se muestra en esta parte tenemos el confirmar con google el cual nos ayudaria para el inicio rapido de seccion y la autenticacion de la cuenta y no se pueda repetir informacion guardando el correo y la contraseña

```
4 Usages
private fun getErrorMessage(exception: Exception): String {
    return when {
        exception.message?.contains( other = "email") == true -> "Correo electrónico inválido"
        exception.message?.contains( other = "password") == true -> "Contraseña incorrecta o muy débil"
        exception.message?.contains( other = "network") == true -> "Error de conexión. Verifica tu internet"
        exception.message?.contains( other = "user-not-found") == true -> "Usuario no encontrado"
        exception.message?.contains( other = "email-already-in-use") == true -> "Este correo ya está registrado"
        exception.message?.contains( other = "weak-password") == true -> "La contraseña debe tener al menos 6 caracteres"
        else -> "Error: ${exception.message ?: "Algo salió mal"}"
    }
}
}
```

En este apartado lo que hace es Esta función se encarga de traducir los errores técnicos de Firebase en mensajes claros para el usuario. Analiza el contenido del error y devuelve textos comprensibles como “correo inválido”, “usuario no encontrado” o “error de conexión”, mejorando la experiencia de uso de la aplicación.



```xml
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools">

    <uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
    <uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION" />

    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.Mapas">

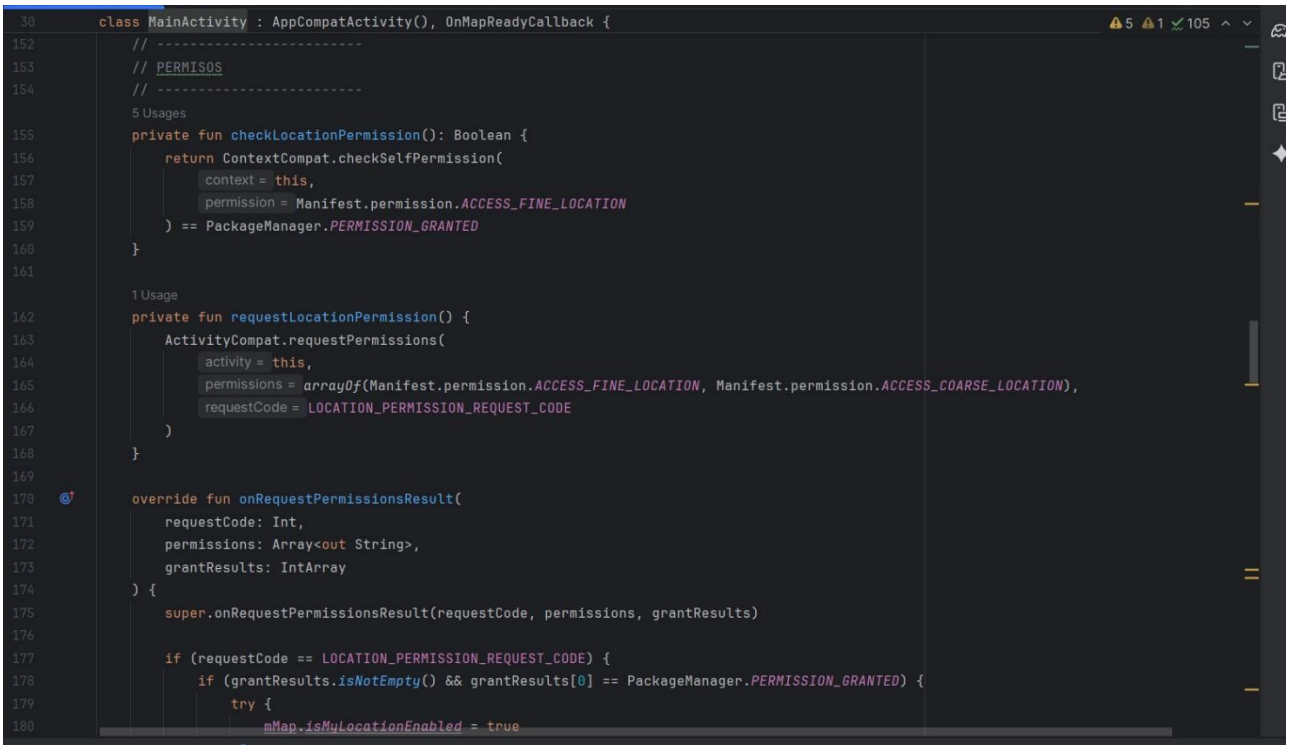
        <meta-data
            android:name="com.google.android.geo.API_KEY"
            android:value="AIzaSyAJqzW9xU-CzomedscA9YgVzbPpdzXsU-w" />

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

Configuración de Permisos y Metadatos (Manifest)

En esta parte lo que podemos ver es la configuración inicial en el AndroidManifest.xml. Se definen los permisos necesarios para que la aplicación funcione correctamente con servicios de red y GPS:

Así mismo, se incluye la API KEY de Google Maps dentro del nodo <meta-data>, la cual sirve para autenticar nuestra aplicación ante los servicios de Google Cloud.



```
class MainActivity : AppCompatActivity(), OnMapReadyCallback {
    // ------------------------
    // PERMISOS
    // ------------------------
    5 Usages
    private fun checkLocationPermission(): Boolean {
        return ContextCompat.checkSelfPermission(
            context = this,
            permission = Manifest.permission.ACCESS_FINE_LOCATION
        ) == PackageManager.PERMISSION_GRANTED
    }

    1 Usage
    private fun requestLocationPermission() {
        ActivityCompat.requestPermissions(
            activity = this,
            permissions = arrayOf(Manifest.permission.ACCESS_FINE_LOCATION, Manifest.permission.ACCESS_COARSE_LOCATION),
            requestCode = LOCATION_PERMISSION_REQUEST_CODE
        )
    }

    override fun onRequestPermissionsResult(
        requestCode: Int,
        permissions: Array<out String>,
        grantResults: IntArray
    ) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults)

        if (requestCode == LOCATION_PERMISSION_REQUEST_CODE) {
            if (grantResults.isNotEmpty() && grantResults[0] == PackageManager.PERMISSION_GRANTED) {
                try {
                    mMap.isMyLocationEnabled = true
```

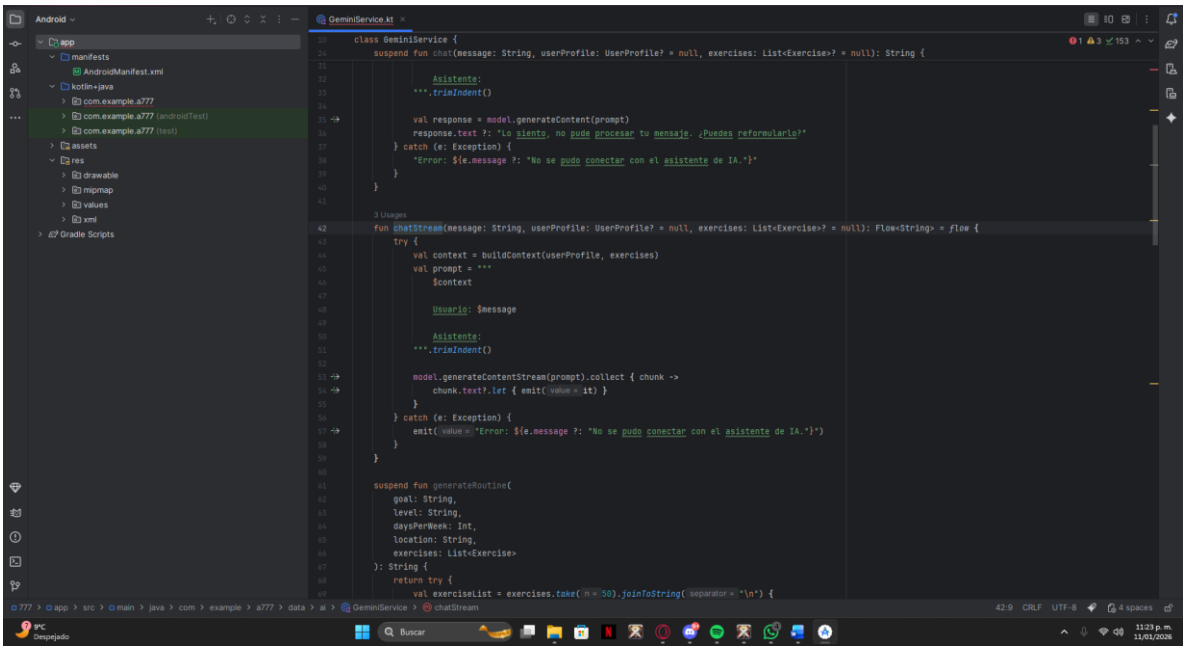
Este apartado es para cuando la aplicación necesita validar si el usuario ha otorgado acceso a su ubicación. Contiene tres funciones clave:

checkLocationPermission: Devuelve un valor booleano indicando si el permiso ya fue concedido.

requestLocationPermission: Lanza el cuadro de diálogo del sistema para pedir los permisos de ubicación fina y gruesa.

onRequestPermissionsResult: Es el encargado de recibir la respuesta del usuario; si el permiso es aceptado, activa la propiedad isMyLocationEnabled para mostrar el punto azul en el mapa.

FILE STREAM



Este apartado describe la lógica encargada de gestionar la comunicación en tiempo real con el modelo Gemini. Utiliza flujos asíncronos para permitir que la aplicación reciba y muestre respuestas de forma progresiva. Contiene tres componentes clave:

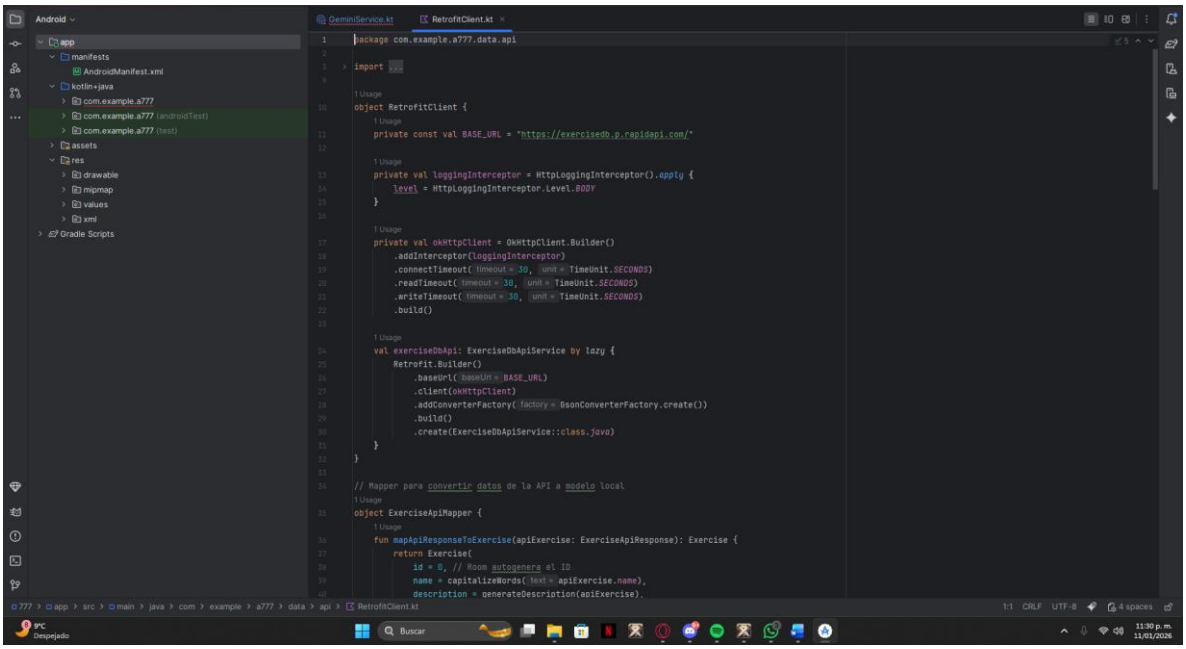
- **chatStream**: Es la función principal que devuelve un objeto Flow<String>. Actúa como un canal que emite fragmentos de texto conforme la IA los genera, permitiendo una experiencia de usuario fluida y dinámica.

```
        model.generateContentStream(prompt).collect { chunk ->
            chunk.text?.let { emit( value = it) }
        }
    } catch (e: Exception) {
        emit( value = "Error: ${e.message ?: "No se pudo conectar con el asistente de IA."}")
    }
}
```

- **generateContentStream**: Se encarga de enviar el contexto del usuario (perfil y ejercicios) al modelo de lenguaje. Utiliza el método collect para capturar cada nuevo dato (chunk) que llega desde los servidores de Gemini y transmitirlo inmediatamente a la interfaz.

- **Manejo de Excepciones (Try-Catch)**: El flujo incluye un control de errores que, en caso de fallas en la conexión o con la API, emite un mensaje amigable indicando que no se pudo conectar con el asistente, evitando que la aplicación se detenga inesperadamente.

WEB SERVICES DML



Este apartado es para cuando la aplicación necesita comunicarse con servidores externos para gestionar la información de los ejercicios. Contiene tres componentes clave:

- **RetrofitClient**: Es el objeto encargado de configurar la conexión de red. Define la URL base, los tiempos de espera (timeouts) y añade un interceptor de registros (logging) para supervisar todas las peticiones y respuestas entre la App y el servidor.

```
    1 Usage
    val exerciseDbApi: ExerciseDbApiService by lazy {
        Retrofit.Builder()
            .baseUrl( baseUrl = BASE_URL)
            .client(okHttpClient)
            .addConverterFactory( factory = GsonConverterFactory.create())
            .build()
            .create(ExerciseDbApiService::class.java)
    }
}
```

- **ExerciseDbApiService**: Define la interfaz de comunicación (Web Service) donde se declaran las operaciones de obtención y manipulación de datos. Es el punto de entrada para solicitar la lista de ejercicios disponibles en la base de datos externa.

```
object ExerciseApiMapper {
    1 Usage
    fun mapApiResponseToExercise(apiExercise: ExerciseApiResponse): Exercise {
        return Exercise(
            id = 0, // Room autogenera el ID
            name = capitalizeWords( text = apiExercise.name),
            description = generateDescription(apiExercise),
            instructions = formatInstructions(apiExercise.instructions),
            animationUrl = apiExercise.gifUrl,
            primaryMuscleGroup = mapTargetToMuscleGroup( target = apiExercise.targetMuscle),
            secondaryMuscleGroups = mapSecondaryMuscles(apiExercise.secondaryMuscles),
            equipment = mapEquipment(apiExercise.equipment),
            difficulty = determineDifficulty(apiExercise),
            category = ExerciseCategory.STRENGTH,
            location = determineLocation(apiExercise.equipment)
        )
    }

    3 Usages
    private fun capitalizeWords(text: String): String {
        return text.split( ...delimiters = " ").joinToString( separator = " ") { word ->
            word.replaceFirstChar { if (it.isLowerCase()) it.titlecase() else it.toString() }
        }
    }
```

- **ExerciseApiMapper**: Es la lógica encargada de transformar los datos. Convierte la respuesta bruta del servidor (ExerciseApiResponse) al modelo de datos interno de la aplicación (Exercise), asegurando que nombres y descripciones tengan el formato correcto antes de ser usados.

CONCLUCIÓN

El desarrollo de la aplicación móvil orientada al ejercicio y el bienestar físico representa una integración significativa entre la tecnología y las necesidades actuales de la sociedad. A lo largo de este proyecto se ha podido observar cómo el uso adecuado de herramientas de desarrollo y tecnologías móviles permite construir soluciones funcionales que impactan directamente en la calidad de vida de las personas. La posibilidad de crear rutinas personalizadas, organizar actividades por días de la semana, gestionar dietas y llevar un control visual del progreso fomenta hábitos saludables y contribuye a una mayor motivación por parte de los usuarios.

Asimismo, la incorporación de funcionalidades como el uso de mapas, GPS y contadores de tiempo convierte a la aplicación en una herramienta más completa, capaz de acompañar al usuario durante su actividad física de manera práctica y dinámica. Estas características no solo enriquecen la experiencia de uso, sino que también permiten obtener información precisa sobre el desempeño personal, lo que favorece la toma de decisiones para mejorar resultados y mantener la constancia en el ejercicio.

Desde el punto de vista académico y técnico, este proyecto fortalece el aprendizaje en el desarrollo de aplicaciones móviles mediante Android Studio y Kotlin, así como la aplicación práctica de conceptos fundamentales como el manejo de bases de datos con SQLite, el uso de almacenamiento interno con SharedPreferences y FileStream, y la interacción con servicios externos mediante Web Services. Cada uno de estos elementos aporta valor al sistema y demuestra la importancia de diseñar aplicaciones bien estructuradas, seguras y orientadas a las necesidades del usuario.

En conclusión, la aplicación desarrollada no solo cumple con el objetivo de ofrecer una herramienta útil para personas interesadas en mejorar su condición física, sino que también refleja el compromiso con la innovación tecnológica y el aprendizaje continuo. Este proyecto sienta las bases para futuras mejoras y ampliaciones, demostrando que la tecnología móvil puede ser un aliado clave en la promoción de estilos de vida saludables y en la creación de soluciones digitales con impacto positivo en la sociedad.

BIBLIOGRAFIAS

Android Developers. (2023). Guía oficial de desarrollo para Android con Kotlin. Google. https://developer.android.com/docs

Codd, E. F. (1990). The relational model for database management: Version 2. Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7th ed.). Pearson.

Google Developers. (2023). Maps SDK for Android documentation. Google. https://developers.google.com/maps/documentation/android-sdk

Kumar, V., & Bhatia, R. (2021). Mobile health applications and their role in promoting healthy lifestyles. International Journal of Health Sciences, 15(3), 45–52.

OpenAI. (2023). Artificial intelligence in mobile applications. OpenAI Documentation. https://platform.openai.com/docs

Pressman, R. S., & Maxim, B. R. (2020). Software engineering: A practitioner’s approach (9th ed.). McGraw-Hill.

Sommerville, I. (2016). Software engineering (10th ed.). Pearson.

World Health Organization. (2022). Physical activity. World Health Organization. https://www.who.int/news-room/fact-sheets/detail/physical-activity